

Clustering Optimization – Megh Patel and Nathan McClure

1. Problem Definition & Real world context

a. Exact Problem Flavor

- i. Partitioning a dataset into k clusters while keeping Within-Cluster Sum of Squares (WCSS) as low as possible. This means a dataset is grouped into k clusters with the goal of having each cluster contain stuff that is as close to the average (center) of that cluster.

b. Is it Polynomial, NP-hard, or NP-precise

- i. NP-Hard because it takes an exponentially large amount of time to find optimal cluster groups. There is no polynomially efficient way to guarantee optimum grouping.

c. Real world application and industries using this problem

- i. Entertainment group their media by genres. For marketing, targeted advertisements send the same ads to the same cluster of shoppers.

2. Algorithm Selection

a. K-means clustering

- i. K-means repeatedly assigns each point to the nearest cluster and then recalculates the average (center) of the cluster until the cluster stops changing.
- ii. **Time complexity**- $\Theta(nkd)$ n =points, k =clusters, d =dimensions, t =iterations
- iii. **Strengths and Weaknesses**
 1. **Strengths**- conceptually simple, easy to implement, and works well on large datasets
 2. **Weaknesses**- only works well with spherical clusters, must choose k ahead of time
- iv. K-means was chosen because it is the standard baseline for optimizing WCSS. By constantly recalculating the center of the clusters we ensure that WCSS is always getting closer and closer to optimal with each iteration.

b. 2 Additional Algorithms

i. Hierarchical Agglomerative (bottom-up) Clustering

1. Hierarchical agglomerative clustering begins with each point as its own cluster. At each step, it merges the two closest clusters based on single, complete, or average linkage criteria. And this continues until all points form one cluster.
2. **Time complexity** - $\Theta(n^2 \log n)$
3. **Strengths and weakness**
 - a. **Strengths** - does not require choosing k in advance, can capture non-spherical cluster structures, and provides a full hierarchy of clusterings.
 - b. **Weaknesses** - much slower than K-means on large datasets, and is sensitive to the chosen linkage method.

4. HAC represents a different clustering approach, using bottom-up merging rather than centroid optimization. It allows comparison between partition-based and hierarchical approaches, especially on datasets where cluster shapes are not spherical.
- ii. **K-Medoids (PAM)**
1. K-Medoids is similar to K-means, but instead of using the mean of points as the cluster center, it chooses an actual data point (medoid) as the center. The algorithm repeatedly assigns each point to its nearest medoid and then tries swapping medoids with non-medoids to reduce total clustering cost.
 2. **Time complexity** - $\Theta(n^2kt)$ n = # of points, k = clusters, and t = iterations.
 3. **Strengths and weakness**
 - a. **Strengths** - more robust to outliers than K-means, works with any distance metric, and conceptually simple and easy to implement.
 - b. **Weaknesses** - slower than K-means, it still requires choosing k ahead of time, and does not scale as well to very large datasets.
 4. It was chosen because it provides a natural comparison to K-means, as both are partition-based and optimize a similar objective.
- iii. **Particle Swarm Optimization (PSO) for Clustering - Nature-based**
1. PSO is inspired by the movement of bird flocks. Each "particle" represents a candidate clustering solution (a set of centroids). Particles move through the search space by combining their best-known position with the global best-known position. Over time, the swarm converges toward high-quality cluster assignments.
 2. **Time complexity** - $\Theta(S \times n \times k \times d \times t)$ S = swarm size, n = points, k = clusters, d = dimensions, t = iterations.
 3. **Strengths and weakness**
 - a. **Strengths** - Can escape local minima that trap K-means, does not require differentiability, and often produces higher-quality clusters than greedy methods.
 - b. **Weaknesses** - slower than K-means due to population-based search, requires tuning swarm size, inertia, and learning coefficients.
 4. PSO was chosen because it allows exploration of optimization-based clustering. It provides a comparison against k-means because both optimize WCSS, but PSO uses global search rather than greedy local updates.

3. Experimental Design

- a. **3 Research questions** focused on resource trade offs (specify input size, parameters to vary, metrics to measure (runtime, memory, solution quality, resource usage), and number of trials per configuration
 - i. **Question 1:** How does runtime and memory usage of K-means, K-medoids, and Hierarchical Agglomerative Clustering scale as input n increases?
 1. **Input Size:** $n \in \{1000, 5000, 10000, 20000\}$, fixed $k=5$, $d=10$.
 2. **Parameters to vary:** Algorithm type: K-medoids, HAC | Linkage for HAC
 3. **Metrics to measure:** Runtime: wall-clock time | Memory usage: peak memory
 4. **Number of trials:** 5 trials per configuration.
 - ii. **Question 2:** For fixed input size, how do K-means, k-medoids, and PSO-based clustering trade off solution quality versus runtime as the number of clusters k increases?
 1. **Input Size:** $n = 10,000$, $d = 10$, $k \in \{3, 5, 10, 20\}$
 2. **Parameters to vary:** Algorithm: K-means, PSO | PSO iterations and swarm size fixed to reasonable defaults.
 3. **Metrics to measure:** Solution quality: WCSS | Runtime: wall-clock time
 4. **Number of trials:** 5 trials per configuration.
 - iii. **Question 3:** For PSO-based clustering, what is the trade-off between solution quality and computational cost as PSO parameters change?
 1. **Input Size:** $n = 10,000$, $d = 10$, $k = 5$
 2. **Parameters to vary:** Swarm size $S \in \{20, 50, 100\}$ | Iterations $t \in \{50, 100, 200\}$
 3. **Metrics to measure:** Solution quality: WCSS | Runtime: wall-clock time | Resource usage: approximate CPU utilization or total function evaluations.
 4. **Number of trials:** 3-5 trials per configuration (depending on time), averaged.

b. **3 Levels for your project**

- i. **Most ambitious goal for your project**
 1. Run all four algorithms on a large dataset and compare their efficiency as the input size is increased
- ii. **One minimal viable experiment that guarantees project feasibility**
 1. Compare K-means vs K-medoid on a small dataset
- iii. OPTIONAL extra ambitious extension

4. **Dataset plan**

- a. **Identify real dataset** (link) OR describe synthetic generation
 - i. **Test Dataset-** Iris ~ 150 points
 1. <https://archive.ics.uci.edu/dataset/53/iris>
 - ii. **Main Dataset-** Spotify ~ 116k points

1. <https://www.kaggle.com/datasets/tomigelo/spotify-audio-features/data>

b. Estimate realistic input size ranges

- i. Small - 1,000-5,000 points
- ii. Medium ~ 10,000 points
- iii. Large 20,000-30,000

c. Discuss feasibility of computation within semester limits

- i. The spotify dataset is the real dataset we will focus on but realistically it will likely be far too large for us to effectively test all algorithms on. We will use the Iris dataset for debugging to make sure our algorithms work and for our actual testing we will use subsets of the spotify dataset in accordance to our input size ranges.

5. Risk Assessment

a. Discuss Roadblocks

i. Technical gaps

1. PSO is definitely unfamiliar, implementation and debugging could take extra time

ii. Time constraint

1. Some algorithms take far longer on large datasets than others and multiple trials will only increase that. Debugging algorithms on large datasets has the potential to eat a lot of time.

iii. Data processing challenges

1. The spotify dataset includes information not needed for our algorithms that will need to be removed prior to clustering

iv. Parameter tuning complexity if any

1. PSO will require choosing swarm size, inertia weight, and learning coefficients
2. HAC's linkage method can change results
3. K-means and K-medoids require preemptively choosing k which will directly affect runtime

v. Team workload distribution

1. One team member can focus on the more classical algorithms like K-means and K-medoids and the other team member will work on HAC and PSO.